

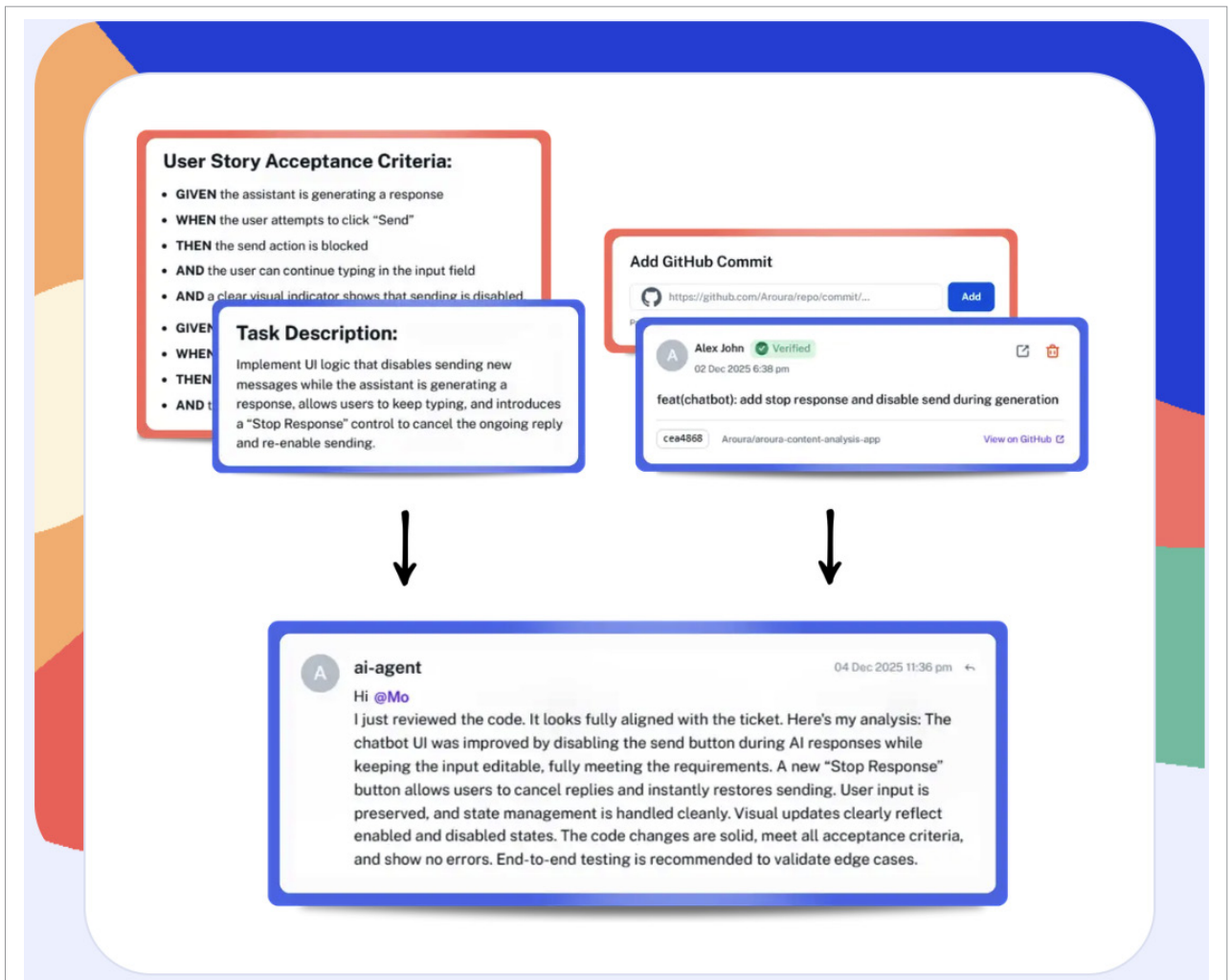


Figure 4: Redefining 'done'

remains unchanged: building reliable, highly available, and maintainable systems. But as software engineering increasingly intersects with data science, the definition of *done* needs to fundamentally evolve.

Passing unit tests, satisfying the linter, and achieving a successful container build is no longer enough. Syntactical correctness does not guarantee behavioural correctness in a data-driven system. Just as a successfully compiled software package is useless if it lacks its required system dependencies, a successfully deployed machine learning service is useless if it misinterprets its incoming data expectations.

For a machine learning feature to truly be ready for a production environment, it must meet a much higher standard rooted in context. It must align with the original intent of the task as defined in the issue tracker. It must respect the data assumptions and feature distributions established during the training phase. And it must behave consistently across both isolated, pristine training containers and chaotic, real-world production environments.

By bringing overarching project context directly into automated pipelines and code reviews, developers can bridge the massive gap between data science and engineering. We can utilise open source principles to build modular, intelligent checks that enforce data contracts automatically. By doing so, we ensure that our machine learning models perform just as flawlessly on a live, production server as they did in a local development environment. **END** 🐧

By: Eyad Ayman

The author is a software and AI engineer experienced in building generative AI applications at Omdena. He specialises in context-aware code review systems and intelligent developer tools, focusing on improving code quality, automation, and AI-assisted software development.